

Vietnam National University, Ho Chi Minh City
University of Technology
Faculty of Computer Science and Engineering



DISCRETE STRUCTURE (CO1007)

Assignment

A * ALGORITHMS

(UPDATE April 28th, 2026)

Instructor(s): Nguyễn An Khương, *CSE-HCMUT*
Lê Hồng Trang, *CSE-HCMUT*
Trần Tuấn Anh, *CSE-HCMUT*
Phạm An Vinh, *CSE-HCMUT*
Mai Xuân Toàn, *CSE-HCMUT*
Trần Hồng Tài, *CSE-HCMUT*

Note: **To reduce the chance and effectiveness of cheating cases, there will be harmony questions in the final exam.**

Table of Contents

1	Introduction	1
1.1	Graph Theory	1
1.2	Path in a Graph	1
1.3	Pathfinding Algorithms	1
1.4	The Shortest Path Problem	2
1.5	A* Algorithm	2
2	Project Requirements	3
2.1	Programming Tasks	3
2.2	Function Naming for Auto Testing	5
2.3	Common Data Structure Requirement	6
3	VERSION INFO & QUESTIONS	6
3.1	Version Info	6
3.2	Guidelines for Asking Questions	7
3.3	FAQ	7
4	Instructions and requirements	7
4.1	Instruction	7
4.2	Submission	7
4.3	Cheating treatment	8

1 Introduction

1.1 Graph Theory

Graph theory is a foundational area of discrete mathematics that studies the relationships between objects using vertices and edges. A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E , where each edge connects a pair of vertices. Graphs can be directed or undirected, weighted or unweighted, and may represent a wide range of systems such as networks, maps, or decision trees. In computer science, graphs are used to model data structures, optimize routes, and analyze connectivity. Directed graphs (digraphs) are particularly useful in pathfinding problems, where each edge has a direction and possibly a cost associated with traversal. Weighted graphs assign numerical values to edges, representing distances, time, or other metrics. Graph theory provides the mathematical framework for designing and analyzing algorithms that operate on these structures, including traversal, search, and optimization techniques. Understanding graph properties—such as connectivity, cycles, and shortest paths—is essential for solving complex problems in fields like artificial intelligence, logistics, and operations research. In this project, we use a directed weighted graph to represent a maze, where each cell is a node and each valid move is an edge.

1.2 Path in a Graph

A path in a graph is a sequence of vertices connected by edges, representing a route from one point to another. Formally, a path from vertex v_0 to v_n is a sequence $v_0, v_1, \dots, v_n$ such that each pair (v_i, v_{i+1}) is an edge in the graph. The length of a path is typically defined as the number of edges it contains, while the cost of a path in a weighted graph is the sum of the weights of its edges. Paths can be simple (no repeated vertices), cyclic (starting and ending at the same vertex), or optimal (minimizing cost or length). In practical applications, paths are used to model navigation, communication, and decision-making processes. For example, in a maze represented as a graph, a path corresponds to a sequence of moves from the entrance to the exit. Identifying valid paths requires checking adjacency and avoiding obstacles or invalid transitions. In optimization problems, the goal is often to find the shortest or least-cost path among many possibilities. Algorithms like Dijkstra's and A* are designed to efficiently explore graphs and identify optimal paths based on specific criteria.

1.3 Pathfinding Algorithms

Pathfinding algorithms are computational techniques used to determine a viable or optimal route between two points in a graph. These algorithms play a crucial role in various domains such as robotics, logistics, computer games, and navigation systems. At their core, pathfinding methods explore the structure of a graph to identify a sequence of edges that connect a start node to a goal node, often while minimizing cost, distance, or time.

In weighted graphs, each edge carries a numerical value representing the cost of traversal. Pathfinding algorithms must account for these weights to avoid inefficient or costly routes. Some algorithms guarantee the shortest path, while others prioritize speed or simplicity, depending on the application. Common strategies include breadth-first search (BFS), depth-first search (DFS), and uniform cost search, each with its own strengths and limitations.

Effective pathfinding depends on how the graph is represented, how neighbors are evaluated, and how decisions are made during traversal. In maze-solving problems, for example, the graph is typically a grid, and the algorithm must navigate around walls, dead ends, and open paths.

The final output is a sequence of nodes forming a valid path from the start to the goal, often reconstructed by tracing back through visited nodes once the goal is reached.

1.4 The Shortest Path Problem

The shortest path problem is a fundamental question in graph theory and computer science, concerned with finding the minimum-cost path between two vertices in a graph. Given a weighted graph $G = (V, E)$, where each edge $(u, v) \in E$ has an associated non-negative weight $w(u, v)$, the goal is to determine a path from a source vertex s to a destination vertex t such that the sum of the edge weights along the path is minimized.

This problem has numerous real-world applications, including GPS navigation, network routing, transportation planning, and robotics. Depending on the graph's structure and constraints, different algorithms may be employed. For instance, Dijkstra's algorithm efficiently solves the single-source shortest path problem in graphs with non-negative weights, while the Bellman-Ford algorithm can handle graphs with negative edge weights. In cases where all edge weights are equal, a breadth-first search (BFS) can be used to find the shortest path in terms of the number of edges.

The shortest path problem also serves as the foundation for more advanced optimization techniques, such as those used in heuristic search, dynamic programming, and reinforcement learning. Understanding this problem is essential for designing intelligent systems that operate efficiently in complex environments.

1.5 A* Algorithm

The A* (A-star) algorithm is a widely used pathfinding and graph traversal technique in computer science and artificial intelligence. It is designed to find the shortest path from a starting node to a goal node in a weighted graph or grid-based environment.

A* combines the strengths of Dijkstra's algorithm and Greedy Best-First Search by using a cost function defined as:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the actual cost from the start node to the current node.
- $h(n)$ is the heuristic estimate of the cost from the current node to the goal.
- $f(n)$ is the total estimated cost of the cheapest solution through node n .

If the heuristic $h(n)$ is admissible (i.e., it never overestimates the true cost to reach the goal), A* is guaranteed to find the optimal path. The choice of a heuristic has a significant impact on the algorithm's efficiency and accuracy.

A* is commonly applied in:

- Maze and map navigation
- Puzzle solving (e.g., 8-puzzle)
- Robotics and autonomous movement
- Game AI and route planning

For example, the graph below illustrates a simple example of the A* pathfinding algorithm. Each node is labeled with its name, heuristic value $h(n)$, and total cost estimate $f(n) = g(n) + h(n)$. In this graph:

- Node **S** is the **start node**.
- Node **G** is the **goal node**.
- The optimal path is highlighted in **red**: **S** → **A** → **D** → **G**.

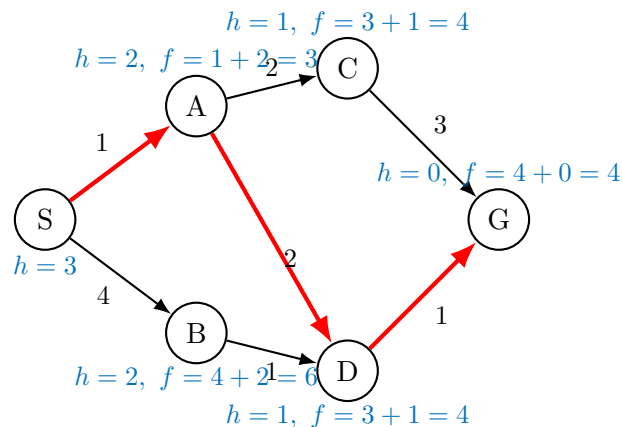


Fig. 1: Example for A* algorithm

In this project, students will implement A* in various contexts and explore its behavior under different heuristic strategies.

2 Project Requirements

2.1 Programming Tasks

Students are required to complete the 4 programming tasks described in this section, each involving the application of the A* search algorithm to solve a practical problem. All implementations must be written in C++, using only the default libraries provided by the LMS CodeRunner environment, which include: `<iostream>`, `<fstream>`, `<string>`, `<cmath>`, `<vector>`, and `<algorithm>`. Finally, students must submit the following:

- `PathNode.h` and `PathNode.cpp` for the `PathNode` struct and related functions.
- `Algo.h` and `Algo.cpp` containing all functions that solve the tasks.
- A `main.cpp` file that demonstrates the program's functionality using sample input and output.
- A report file that answers the report requirements for each task, named after your student ID (e.g., 24252628.pdf).

1. **Degrees of Separation in a Social Network (2.5 pts):** Given a social network graph represented by an adjacency matrix, implement the A* algorithm to find the shortest connection path between two people.

- **Input:** Adjacency matrix, start person, goal person.

- **Output:** A linked list containing the path of people from start to goal, along with $f(n)$, $g(n)$, and $h(n)$ values at each step.
 - **Cost Function:**
 - $g(n)$: Number of connections traversed from the start person to the current person.
 - $h(n)$: Estimated minimum number of connections remaining to reach the goal.
 - $f(n) = g(n) + h(n)$
 - **Report Requirement:** Explain how $h(n)$ is calculated and provide a manual example run.
2. **Drone Delivery in 2D Space with Three Heuristics (2.5 pts):** Given a set of delivery points represented by coordinates (x, y) and weighted edges, implement A* to find the shortest delivery route between two points using three heuristic modes.
- **Input:** Weight matrix, list of coordinates, start point, goal point, heuristic mode.
 - **Output:** A linked list containing the path, with $f(n)$, $g(n)$, and $h(n)$ values at each vertex.
 - **Cost Function:**
 - $g(n)$: Total travel cost from the start point to the current point, calculated as the sum of edge weights along the path.
 - $h(n)$: Estimated cost from the current point to the goal point using the selected distance metric.
 - $f(n) = g(n) + h(n)$
 - **Heuristic Modes:**
 - Mode 1: Manhattan distance — $h(n) = |x_1 - x_2| + |y_1 - y_2|$
 - Mode 2: Euclidean distance — $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
 - Mode 3: Chebyshev distance — $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$
 - **Report Requirement:** Compare the three heuristics with manual example runs where each heuristic has its advantage.
3. **Warehouse Robot Navigation with Obstacles (2.5 pts):** Given a warehouse layout as an $m \times n$ grid, where each cell is either passable (0) or blocked (1), implement A* to find the shortest path for a robot from a start coordinate to a goal coordinate.
- **Input:** Grid layout, start coordinate, goal coordinate, heuristic mode.
 - **Output:** A linked list containing the sequence of moves with names chosen from the following set:
 - Cardinal directions: Up, Down, Left, Right
 - Diagonal directions: Up-Left, Up-Right, Down-Left, Down-Right
 Each node in the linked list should be named according to the direction taken (e.g., "Down" or "Up-Right"), and must include the values of $f(n)$, $g(n)$, and $h(n)$.
 - **Cost Function:**
 - $g(n)$: Total cost from the start node to the current cell. - Horizontal and vertical moves cost 1 unit. - Diagonal moves cost 1.5 units. Thus, $g(n)$ accumulates the sum of these move costs along the path.

- $h(n)$: Estimated cost to the goal, calculated using the selected heuristic mode:
 - * Mode 1: Manhattan distance — $h(n) = |x_1 - x_2| + |y_1 - y_2|$
 - * Mode 2: Chebyshev distance — $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$
 - $f(n) = g(n) + h(n)$
 - **Report Requirement:** Provide a manual example run with a small warehouse grid, showing the chosen directions and the computed $g(n)$, $h(n)$, and $f(n)$ at each step and comparing both heuristic modes.
4. **Evacuation Route Planning (2.5 pts):** Given a building floor plan represented as an $m \times n$ grid, rebuild it as a graph using a weight matrix. Each tile is treated as a vertex. Implement A* to find the shortest evacuation route from a start coordinate to an exit.
- **Input:** Floor plan grid, start coordinate, exit coordinate, empty weight matrix, heuristic mode.
 - **Output:** A linked list representing the path with $f(n)$, $g(n)$, and $h(n)$ values, plus the final weight matrix.
 - **Cost Function:**
 - $g(n)$: Total cost from the start node to the current vertex. - Horizontal and vertical moves cost 1 unit. - Diagonal moves cost 1.5 units. Thus, $g(n)$ accumulates the sum of these move costs along the path.
 - $h(n)$: Estimated cost to the exit, calculated using the selected heuristic mode:
 - * Mode 1: Manhattan distance — $h(n) = |x_1 - x_2| + |y_1 - y_2|$
 - * Mode 2: Chebyshev distance — $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$
 - $f(n) = g(n) + h(n)$
 - **Report Requirement:** Describe the conversion process from grid to weight matrix and provide a manual example run comparing both heuristic modes.

2.2 Function Naming for Auto Testing

To facilitate automated testing, each programming task must be implemented as a function with the following standardized name and signature. The auto tester will use these function names to validate correctness.

- **Task 1 — Degrees of Separation in a Social Network:**
`PathNode* findSocialPath(double adjMatrix[100][100], int startPerson, int goalPerson);`
- **Task 2 — Drone Delivery in 2D Space with Three Heuristics:**
`PathNode* findDronePath(double adjMatrix[100][100], int coords[100][2],
 int startPoint, int goalPoint, int mode);`
- **Task 3 — Warehouse Robot Navigation with Obstacles:**
`PathNode* findWarehousePath(int warehouse[100][100], int m, int n, int startX,
 int startY, int goalX, int goalY, int mode);`
- **Task 4 — Evacuation Route Planning:**
`PathNode* findEvacuationPath(int floorPlan[100][100], int m, int n, int startX,
 int startY, int exitX, int exitY, double weightMatrix[100][100], int mode);`

Each function must return a pointer to the head of a linked list of **PathNode** objects, representing the path from start to goal. The list should include the direction (e.g., Up, Down, Left, Right, Up-Left, Up-Right, Down-Left, Down-Right) or coordinate name, along with the cost values ($f(n)$, $g(n)$, $h(n)$) for each step.

2.3 Common Data Structure Requirement

For all four problems described in this project, students must define and use a common linked list data structure to represent the solution path. Each node in the linked list must contain the following fields:

- **Name:** a `std::string` representing the name or label of the node (e.g., vertex name, coordinate).
- $f(n)$: the total estimated cost from the start node to the goal through this node.
- $g(n)$: the actual cost from the start node to this node.
- $h(n)$: the heuristic estimate from this node to the goal.
- **Next:** a pointer to the next node in the path.

Example (C++ struct):

```
struct PathNode {  
    std::string name;  
    double f;  
    double g;  
    double h;  
    PathNode* next;  
};
```

This linked list should be used to store and output the final path found by the A* algorithm in each problem, preserving the order from the start node to the goal node. The following `printPath` function will be included in the LMS test case system for scoring. Make sure your struct works with it and **don't include this function in your LMS code runner submission**.

```
void printPath(PathNode* head) {  
    cout << "Solution Path:\n";  
    while (head != nullptr) {  
        cout << "Node: " << head->name  
            << " | f: " << head->f  
            << " | g: " << head->g  
            << " | h: " << head->h << "\n";  
        head = head->next;  
    }  
}
```

3 VERSION INFO & QUESTIONS

3.1 Version Info

- First Version

3.2 Guidelines for Asking Questions

If you have any questions about the assignment, please use the official Google Form:

<https://forms.gle/aQd6spSQMK3Hux9H8>

Before submitting a question, you must first check whether it has already been answered in:

- The FAQ section (below),
- Other sections of this assignment document (instructions, requirements, submission rules, etc.), or
- The shared google sheet: [Questions Response Sheet](#)

Important Penalty Rule: Submitting a question that has already been answered in this document, the FAQ, or the answer sheet will result in a penalty point.

3.3 FAQ

- None yet

4 Instructions and requirements

Students have to follow the instructions and comply with the requirements below. *Lecturers do not solve the cases arising because students do not follow the instructions or do not comply with the requirements.*

4.1 Instruction

- **Deadline:** All submissions must be completed by May 31th, 2026. Late submissions will not be accepted.
- **Programming Language:** All tasks must be implemented in C++.
- **Clarity:** Answers must be written clearly and coherently in both code and report form.

4.2 Submission

To receive a score, students must complete **both** submission steps:

1. **Auto-testing code submission:** Submit your code to the designated auto-testcase runner on the LMS site. Each task has its own section in the submission interface. *Note: The auto-testing submission portal will be opened later.*
2. **File submission:** Prepare all required files listed in Section 2.1. Place them into a folder named after your student ID (e.g., 2121323). Compress this folder into a **.rar** file with the same name (e.g., 2121323.rar). Submit this compressed file to the LMS file submission portal.
 - The file submission portal will be available only on the final day of the deadline.
 - **Penalties:** Each of the following errors will result in a **-1 point deduction per occurrence:**

- Missing required file
- Incorrect file name
- Incorrect folder name
- Incorrect compression format (must be .rar)
- Disorganized file structure

4.3 Cheating treatment

This project must be completed **individually**. Collaboration is limited to general discussion of concepts; all submitted code, reports, and explanations must be your own work. Any violation of these rules will be treated as academic misconduct under university regulations. A student will be considered to have committed cheating if:

- **Unusual similarity of submissions:** If two or more students submit work with significant overlap in code, structure, or written content, **all parties** will be considered to have cheated. Protect your work and do not share files or solutions.
- **Lack of understanding of submitted work:** If a student cannot explain the meaning, logic, or reasoning behind their own code or report when questioned, the submission will be treated as dishonest. You may consult textbooks, online resources, or discussions, but you must write and implement the ideas in your own words and code.
- **Unauthorized copying:** Directly copying solutions from peers, online sources, or AI tools without proper adaptation and understanding is prohibited.

Important: Students are responsible for safeguarding their work. Sharing your files or allowing others to copy your solutions is considered cheating, even if you were not the one who copied. All cases of cheating will be handled according to the university's disciplinary regulations.